

Instagram Authentication Project

Detailed Architecture and Integration Report

Generated: 2026-03-03 23:31:02

1. Where TypeScript, JavaScript, and HTML are merged

1.1 Web app bootstrap flow

- client/index.html loads /src/main.tsx
- client/src/main.tsx mounts React Router + App component
- client/src/App.tsx defines route map for all pages

1.2 TypeScript (frontend) to JavaScript (backend) bridge

- TS pages/hooks call fetch/apiRequest to /api endpoints
- client/src/lib/queryClient.ts centralizes token header + request wrapper
- client/src/hooks/use-analysis.ts wraps /api/analyses for React Query
- server/index.js receives requests and persists analysis data

1.3 Vite runtime merge layer

- client/vite.config.ts sets alias @ -> src
- dev server proxy maps /api to localhost:5000 backend
- Browser sees one app, but it is TS UI + JS API + HTML shell

1.4 Extension runtime merge layer

- HTML/JS popup UI lives in client/public/popup.html + popup.js
- manifest.json binds popup, background.js, and content scripts
- Extension shares backend API with main app through localhost endpoints

2. How client analysis is merged

2.1 User profile flow from Analyze page

- client/src/pages/AnalyzePage.tsx parses scraped profile JSON
- analyzeHeuristics(profile) runs client-side scoring
- result is posted to /api/analyses/client with auth token

2.2 Server merge endpoint

- server/index.js /api/analyses/client merges:
 - a) scraped profile metrics
 - b) client heuristic payload
 - c) Python model output (when available)
 - d) calibration and evidence guards

2.3 Scoring pipeline components

- extractProfileMetrics(...) normalizes profile features
- calibrateProfileRiskScore(...) calibrates raw model score
- applyProfileRiskGuards(...) caps risk when evidence is weak

- deriveClassificationTag(...) maps risk+signals to category labels

2.4 Persistence and dashboard consumption

- writes profile_analysis and final_predictions rows
- dashboard and history consume these rows via /api/dashboard and /api/analyses

3. How machine learning is connected

3.1 Node -> Python bridge

- server/index.js uses child_process.spawn('python', [script, payload])
- runPythonScript(...) captures stdout JSON response

3.2 Profile model integration

- buildProfileModelFeatures(...) engineers expected feature columns
- getProfilePrediction(...) calls server/ml/predict.py
- predict.py loads primary RF model, fallback XGBoost model

3.3 Message model integration

- getMessagePrediction(...) calls server/ml/predict_messages.py
- predict_messages.py loads vectorizer + NB/LR model
- outputs calibrated risk_score, mean_probability, threshold_used

3.4 Auto-training hooks

- ensureProfileModelsTrained() and ensureMessageModelsTrained()
- if model files are missing and datasets exist, training scripts run

3.5 Runtime environment requirement

- backend uses the 'python' command from PATH
- project uses vir/Scripts/python.exe to keep model deps stable

4. Extension mode creation and control (step-by-step)

4.1 Extension definition

- manifest_version 3 in client/public/manifest.json
- service worker: background.js
- popup UI: popup.html + popup.js
- Instagram + dashboard content scripts configured

4.2 Popup modes

Buttons in popup.html:

- Final Prediction (Combined)
- Final Prediction (Profile Only)
- Final Prediction (Message Only)
- Analyse Profile
- Analyse Messages

4.3 Mode handlers

In popup.js:

- finalPredictionBtn -> performFinalPrediction({mode:'combined'})
- finalProfileOnlyBtn -> performFinalPrediction({mode:'profileOnly'})
- finalMessageOnlyBtn -> performFinalPrediction({mode:'messageOnly'})

4.4 Data collection stage

- Popup sends ANALYZE_PAGE / ANALYZE_MESSAGES to active tab
- instagram_analyzer.js collects profile or message data
- On DM pages, analyzer delegates background-tab analysis via background.js

4.5 Deep/server analysis stage

- popup.js posts profile heuristic payload to /api/analyses/client
- popup.js posts message payload to /api/analyses
- popup.js fetches /api/analyses/:id for deep result details

4.6 Final synthesis stage

- performFinalPrediction() computes includeProfile/includeMessages
- applies combined/fallback logic and category mapping
- shows status, risk level, confidence, reasons in popup UI

4.7 Session synchronization between web dashboard and extension

- Login page emits window.postMessage(INSTAGRAM_AUTHENTICATION_USER_LOGGED_IN)
- dashboard_content.js forwards user/token to background.js
- background.js stores session in chrome.storage.local
- popup.js reads token from chrome.storage.local for API requests

5. Important practical notes

- You have two frontends: React web dashboard and Chrome extension popup.
- Both call the same backend API.
- Profile-only mode can run with incomplete message evidence.
- Combined mode can fall back to profile-only or message-only when one side is missing.
- New evidence-gating logic avoids over-classifying low-data accounts as high-risk.

6. Key files map

Frontend (TypeScript/React)

- client/index.html
- client/src/main.tsx
- client/src/App.tsx
- client/src/pages/AnalyzePage.tsx
- client/src/lib/queryClient.ts
- client/src/hooks/use-analysis.ts
- client/vite.config.ts

Backend (JavaScript/Node)

- server/index.js
- server/scrapper.js

Machine Learning (Python)

- server/ml/predict.py
- server/ml/predict_messages.py
- server/ml/train_model.py
- server/ml/train_message_models.py

Extension (HTML/JavaScript)

- client/public/manifest.json
- client/public/popup.html
- client/public/popup.js
- client/public/background.js
- client/public/instagram_analyzer.js
- client/public/dashboard_content.js